Contents

Chapter 14: Architecture Review

Week 1 was about building an AI application.

Chapter 14 is about something more important:

> **Learning to reason about the application as a system.**

A developer can make the application work.

An AI systems engineer must be able to explain:

- what the system is,
- why it is structured this way,
- what its interfaces are,
- what can fail,
- what can be attacked,
- what it costs,
- how it performs,
- how it is evaluated,
- and what guarantees it provides.

This is the transition from **implementation thinking** to **systems thinking**.

The application you built during Week 1—the Personal Research Assistant—is now treated as a production candidate rather than a coding exercise.

The question changes from:

> "Does it work?"

to:

> **"Can we defend this architecture?"**

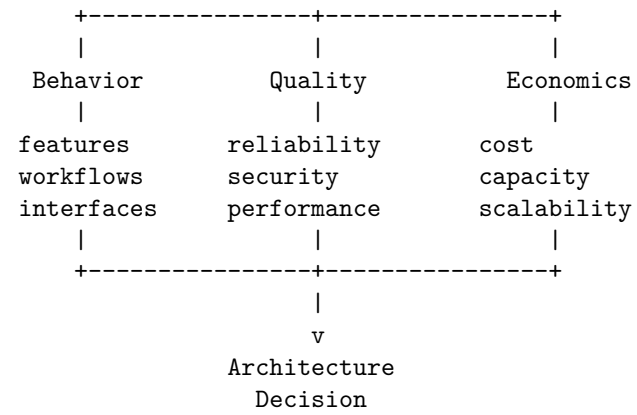
---

1. What Is an Architecture Review?

An architecture review is a structured examination of a system against its requirements, constraints, risks, and operating environment.

A useful abstraction is:

```
System Specification
        |
        v
Architecture
        |
```



The architecture review asks whether the design provides an acceptable solution across all these dimensions.

It should identify:

- architectural assumptions
- system boundaries
- dependencies
- bottlenecks
- failure domains
- security boundaries
- cost drivers
- operational risks
- missing requirements
- technical debt
- unresolved design decisions

An architecture review is therefore not a presentation.

It is a **risk-discovery mechanism**.

---

2. Architecture Is a Set of Tradeoffs

There is rarely one objectively correct architecture.

Consider two possible designs.

Design A

```
Application
    ↓
Hosted LLM API
    ↓
Cloud database
```

Advantages:

- simple
- fast to develop
- low operational burden

Disadvantages:

- external dependency
- variable inference cost
- data leaves the environment
- provider availability becomes a dependency

Design B

```
Application
    ↓
Self-hosted inference
    ↓
Local vector database
    ↓
Local document store
```

Advantages:

- greater control
- predictable data locality
- potentially lower marginal inference cost

Disadvantages:

- hardware requirements
- operational complexity
- model serving
- upgrades
- monitoring
- capacity planning

Neither is universally better.

Architecture is therefore fundamentally an optimization problem:

$$A^* = \arg\max_A Utility(A)$$

subject to constraints involving:

$$Cost,\ Latency,\ Reliability,\ Security,\ Quality,\ Complexity$$

The architecture review makes those tradeoffs explicit.

---

3. The Week 1 Application

Recall the Personal Research Assistant:

```
User
  ↓
Application
  ↓
Document ingestion
  ↓
Retrieval
  ↓
Context construction
  ↓
LLM
  ↓
Tool use
  ↓
Evaluation
  ↓
Cited answer
```

The application can:

- ingest documents
- retrieve relevant information
- answer questions
- cite sources
- use tools
- maintain conversational state
- detect uncertainty
- produce structured outputs
- evaluate its own behavior

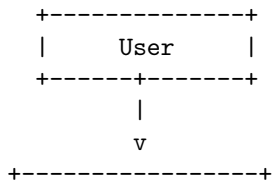
Now we turn that prototype into an architecture.

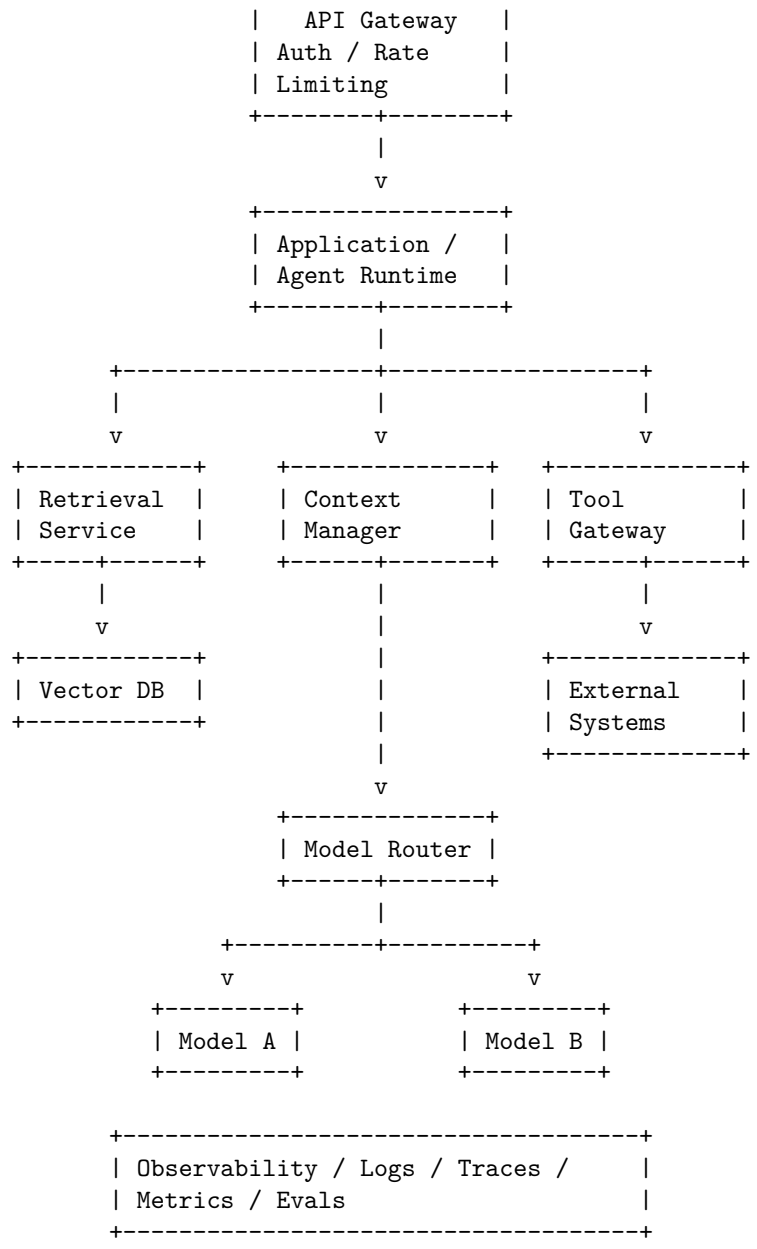
---

4. Deliverable 1 — Architecture Diagram

The first deliverable is the system architecture.

A reasonable production architecture might look like:





The diagram should answer:

> **What are the components, and how does information flow between them?**

It should also show trust boundaries and external dependencies.

A good architecture diagram is not merely decorative.

It is a compressed representation of the system's behavior.

---

5. Architecture Views

One diagram is often insufficient.

A mature architecture review uses multiple views.

Logical view What components exist?

```
Agent
Retriever
Context Manager
Model Router
Tool Gateway
Evaluator
```

Deployment view Where do they run?

```
Client
   ↓
Cloud
   ↓
Application servers
   ↓
Inference infrastructure
   ↓
Databases
```

Data-flow view Where does information move?

```
Document
 ↓
Parser
 ↓
Chunker
 ↓
Embedding
 ↓
Vector DB
 ↓
Retriever
 ↓
LLM
```

Security view Where are the trust boundaries?

```
User
 ↓
Untrusted input
 ↓
Policy boundary
 ↓
Agent
 ↓
Tool authorization
 ↓
External system
```

Architecture review should ensure these views are mutually consistent.

---

6. Architecture Boundaries

One of the most important architecture questions is:

> **Where are the boundaries?**

For example:

```
                 Trust Boundary
                       |
User ------------------|
                       |
                 Application
                       |
                 Policy Boundary
                       |
                    Agent
                       |
                Tool Boundary
                       |
                External System
```

Boundaries determine where we enforce:

- authentication
- authorization
- validation
- isolation
- rate limits
- observability
- failure handling

AI systems introduce unusual boundary problems because natural-language instructions can cross boundaries.

A document retrieved from the Internet may contain text that looks like instructions.

The architecture must distinguish:

`DATA`

from:

`INSTRUCTIONS`

even when both arrive as text.

---

7. Architecture Review Questions

For every component, ask:

Responsibility What does this component own?

Interface What does it expose?

Dependencies What does it depend on?

Failure mode What happens when it fails?

Security boundary What can it access?

Scaling behavior What happens under load?

Cost What does it cost?

Observability How do we know it is working?

This can be represented as:

Component	Responsibility	Dependency	Failure	Security	Scaling	Cost
API	requests	auth service	timeout	public boundary	horizontal	low
Retriever	search	vector DB	empty results	document ACL	horizontal	medium
Model	generation	provider/GPU	unavailable	restricted	capacity-bound	high

Component	Responsibility	Dependency	Failure	Security	Scaling	Cost
Tool gateway	external actions	APIs	failure	privileged	limited	variable
Evaluator	quality	evaluator model	false score	internal	batch	medium

This table often exposes architectural weaknesses faster than the diagram.

---

8. Deliverable 2 — API Specification

The architecture needs explicit interfaces.

An API is a contract between components.

For example:

`POST /v1/query`

Request:

```
{
  "query": "What were the company's revenues in 2025?",
  "conversation_id": "abc123",
  "options": {
    "citations": true
  }
}
```

Response:

```
{
  "answer": "Revenue was ...",
  "citations": [
    {
      "document": "annual_report.pdf",
      "page": 42
    }
  ],
  "confidence": 0.91
}
```

The specification should define:

- request schema
- response schema
- authentication

- authorization
- errors
- timeouts
- rate limits
- idempotency
- versioning
- pagination
- compatibility

For AI systems, also define:

- maximum input size
- maximum context
- model selection
- tool permissions
- output guarantees
- uncertainty representation

---

9. APIs Are Contracts

Consider an agent calling a tool:

```
{
  "tool": "search_documents",
  "arguments": {
    "query": "2025 revenue",
    "limit": 10
  }
}
```

The tool interface should specify:

$$Input \rightarrow Output$$

including failure behavior.

For example:

```
200 → results
400 → invalid request
401 → unauthenticated
403 → unauthorized
429 → rate limited
500 → tool failure
504 → timeout
```

This matters because the agent must know what to do next.

An unspecified error becomes an unpredictable behavior.

A specified error becomes part of the workflow.

---

10. AI APIs Need Behavioral Contracts

Traditional APIs specify structure.

AI APIs also need behavioral expectations.

For example:

```
The assistant MUST:
- cite retrieved sources for factual claims
- distinguish evidence from inference
- refuse unauthorized tool calls
- return structured output
- indicate when evidence is insufficient
```

These are not conventional type constraints.

They are **behavioral contracts**.

Some can be enforced deterministically.

Others require evaluation.

This creates a layered contract:

```
API contract
     +
schema contract
     +
policy contract
     +
behavioral contract
```

---

11. Deliverable 3 — Data Model

Next define what the system stores.

A simple data model might contain:

```
User
 +-- id
 +-- preferences
 +-- permissions

Document
 +-- id
```

```
 +-- owner_id
 +-- metadata
 +-- content

Chunk
 +-- id
 +-- document_id
 +-- text
 +-- embedding
 +-- metadata

Conversation
 +-- id
 +-- user_id
 +-- state

Message
 +-- id
 +-- conversation_id
 +-- role
 +-- content
 +-- timestamp

ToolExecution
 +-- id
 +-- conversation_id
 +-- tool
 +-- arguments
 +-- result
 +-- status
```

The model should capture ownership and authorization.

For example:

$$Document.owner_id = User.id$$

and retrieval should enforce:

$$Accessible(u, d)$$

before returning document (d) to user (u).

---

12. Separate State From Context

This is an important architectural distinction.

State Information the system persists.

```
conversation_id
user_id
document_id
permissions
workflow status
```

Context Information supplied to the model for a particular inference step.

```
system instructions
retrieved documents
conversation summary
tool results
current request
```

The architecture should not blindly persist everything that enters the context.

Instead:

```
Persistent State
       ↓
Context Selection
       ↓
Context Construction
       ↓
Model
```

This makes context management explicit and controllable.

---

13. Data Lifecycle

The review should trace data through its lifecycle:

```
Ingest
  ↓
Parse
  ↓
Store
  ↓
Index
  ↓
Retrieve
  ↓
Context
  ↓
Model
  ↓
```

```
Output
  ↓
Store / Delete
```

At every transition ask:

- Is the data encrypted?
- Who can access it?
- How long is it retained?
- Can it leave the system?
- Is it logged?
- Is it included in model prompts?
- Can it contain malicious instructions?
- Can it be deleted?

This is where architecture, security, privacy, and compliance intersect.

---

14. Deliverable 4 — Threat Model

Security should not be an afterthought.

Perform a formal threat model.

Start with assets.

Assets

```
documents
user data
credentials
API keys
model access
tool permissions
conversation history
```

Then identify actors.

Actors

```
normal user
malicious user
malicious document author
compromised tool
external attacker
insider
```

Then identify attack surfaces.

```
API
uploads
retrieval
prompt
tools
external APIs
dependencies
model output
```

Now construct attack paths.

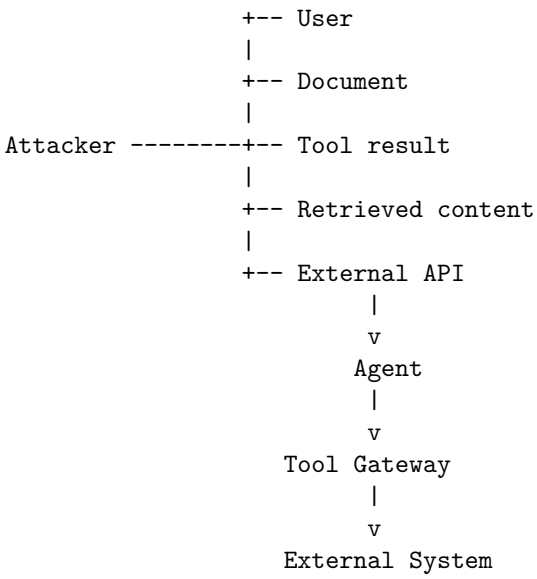
---

15. AI-Specific Threat Model

A traditional application might have:

```
Attacker
   ↓
API
   ↓
Application
```

An agent introduces additional attack paths:



The attacker may never directly compromise the application.

They may instead manipulate information that the agent consumes.

This creates:

Indirect prompt injection.

The architecture must therefore treat external content as untrusted.

---

16. Threat Modeling Agents

For every tool ask:

```
What can this tool do?
Who can invoke it?
What arguments can it accept?
What data can it access?
What side effects can it create?
Can the agent invoke it autonomously?
What approval is required?
```

A dangerous architecture is:

```
LLM
 ↓
arbitrary shell
```

A safer architecture is:

```
LLM
 ↓
Policy engine
 ↓
Tool authorization
 ↓
Argument validation
 ↓
Sandbox
 ↓
External system
```

The architecture review should explicitly identify every privileged capability.

---

17. Threat Modeling Method

For each threat, document:

```
Threat
 ↓
Attack vector
 ↓
Asset affected
```

```
 ↓
Likelihood
 ↓
Impact
 ↓
Mitigation
 ↓
Residual risk
```

For example:

Threat	Vector	Impact	Mitigation
Prompt injection	malicious document	agent compromise	content isolation
Data leakage	model output	sensitive data exposure	access control + filtering
Tool abuse	malicious instruction	external side effect	authorization gateway
API compromise	stolen key	system access	secret management + rotation
Supply-chain attack	dependency	arbitrary code	pinning + scanning

The goal is not to eliminate all risk.

The goal is to understand and control it.

---

18. Deliverable 5 — Cost Model

Now apply Chapter 12.

Estimate:

$$C_{system} = C_{model} + C_{retrieval} + C_{storage} + C_{compute} + C_{network} + C_{observability}$$

For inference:

$$C_{model} = N_{requests}(T_{input}P_{input} + T_{output}P_{output})$$

For an agent:

$$C_{task} = \sum_{i=1}^{k} C_i$$

where k is the number of model/tool operations.

Build assumptions.

For example:

```
10,000 requests/day

Average:
4,000 input tokens
600 output tokens
2.5 model calls/request
```

Then estimate monthly consumption.

The important point is to identify the **cost drivers**.

Perhaps 80% of cost comes from:

```
long context
```

rather than:

```
output generation
```

That changes the optimization strategy.

---

19. Cost Sensitivity Analysis

Do not produce only one cost estimate.

Produce a range.

For example:

```
                  Monthly Cost

Low usage          $300
Expected           $1,200
High usage         $5,000
Extreme            $20,000
```

Then ask:

> What happens if usage doubles?

or:

> What happens if average context increases 50%?

This is sensitivity analysis.

Mathematically:

$$\frac{\partial C}{\partial T_{input}}$$

tells us how sensitive cost is to input tokens.

Similarly:

$$\frac{\partial C}{\partial N_{requests}}$$

tells us how strongly cost scales with traffic.

This turns cost from an accounting number into an architectural variable.

---

20. Deliverable 6 — Reliability Model

Now ask:

> **What happens when components fail?**

Consider:

```
API
 ↓
Retriever
 ↓
Vector DB
 ↓
Model
 ↓
Tool
 ↓
External API
```

Each component can fail.

A naive architecture assumes:

```
everything works
```

A systems architecture assumes:

```
everything eventually fails
```

The reliability model identifies:

- failure domains
- dependencies
- retry behavior
- timeouts
- fallback paths
- degradation strategies
- recovery mechanisms

---

21. Failure Dependency Graph

Suppose:

```
Application
    |
    +-- Vector DB
    |
    +-- Model Provider
    |
    +-- Tool API
```

The system's availability depends on those dependencies.

If every dependency must be available:

$$A_{system} = A_1 A_2 A_3$$

If:

```
A1 = 99.9%
A2 = 99.5%
A3 = 99.9%
```

then:

$$A_{system} \approx 0.999 \times 0.995 \times 0.999$$

which is approximately:

$$99.3\%$$

The more dependencies you introduce, the more carefully you need to design failure handling.

---

22. Graceful Degradation

Not every failure should cause total failure.

Suppose retrieval fails.

A robust assistant might respond:

```
Retrieval unavailable.
I cannot verify the answer against your documents.
```

rather than hallucinating.

Similarly:

```
Model A unavailable
       ↓
Model B
```

```
Tool unavailable
       ↓
Answer without tool

Evaluator unavailable
       ↓
Serve request but mark evaluation pending
```

The architecture should explicitly define degraded modes.

For each dependency:

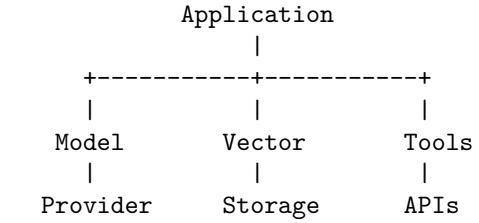
> What is the minimum useful behavior if this component disappears?

---

23. Failure Domains

Identify independent failure domains.

For example:



If all three components depend on the same infrastructure, they may not actually be independent.

A good architecture review asks:

> **What failures can happen simultaneously?**

This prevents false assumptions about redundancy.

---

24. Agent Reliability

Agentic systems introduce another failure mode:

```
runaway execution
```

For example:

```
LLM
 ↓
Tool
 ↓
```

```
LLM
 ↓
Tool
 ↓
LLM
 ↓
Tool
 ↓
...
```

Every agent needs explicit termination conditions:

```
max_steps
max_cost
max_time
max_tool_calls
max_context
```

The runtime should enforce these limits outside the model.

For example:

$$Steps \leq S_{max}$$

$$Cost \leq C_{max}$$

$$Time \leq T_{max}$$

The agent is not allowed to negotiate these limits.

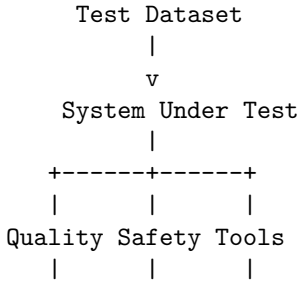
---

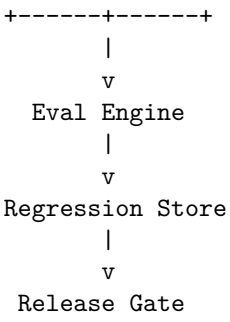
25. Deliverable 7 — Evaluation Strategy

The architecture review must explain:

How will we know that the system works?

The evaluation architecture might be:





Define:

Functional metrics

```
task success
answer correctness
```

Retrieval metrics

```
Recall@k
Precision@k
```

Generation metrics

```
groundedness
citation accuracy
completeness
```

Agent metrics

```
tool accuracy
steps/task
failure recovery
```

System metrics

```
latency
cost
availability
```

Safety metrics

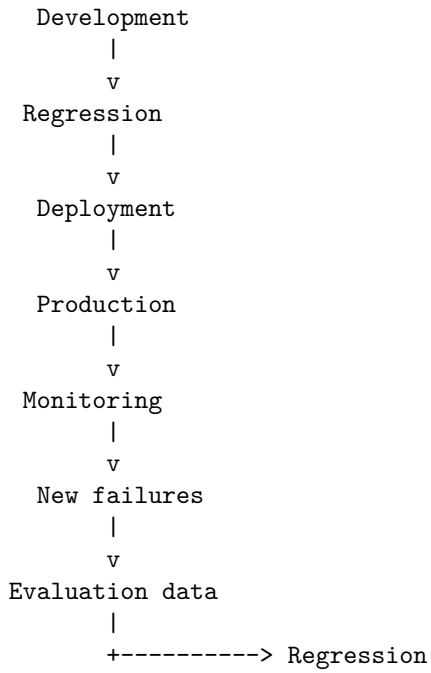
```
attack success rate
unsafe action rate
data leakage rate
```

---

26. Evaluation as an Architectural Component

Evaluation should not be treated as a notebook someone runs occasionally.

It belongs in the architecture:



This creates a self-reinforcing engineering process.

Production failures improve the evaluation suite.

The evaluation suite prevents those failures from returning.

---

27. Deliverable 8 — Performance Model

Finally, model system performance.

Decompose latency:

$$L_{total} = L_{API} + L_{retrieval} + L_{context} + L_{model} + L_{tools}$$

For an agent with sequential model calls:

$$L_{total} = \sum_{i} L_i$$

For parallel operations:

$$L_{parallel} = \max(L_1, \ldots, L_n)$$

Define:

- expected throughput
- peak throughput
- concurrency
- P50 latency
- P95 latency
- P99 latency
- TTFT
- token throughput
- memory requirements

Then identify bottlenecks.

---

28. Capacity Planning

Suppose the system needs:

```
100 requests/sec
```

and each request requires:

```
2,000 input tokens
500 output tokens
```

Then the system requires approximately:

$$200,000$$

input tokens/sec and:

$$50,000$$

output tokens/sec.

This is a fundamentally different engineering problem from:

```
10 requests/minute
```

Capacity planning therefore starts from workload assumptions.

Define:

```
average load
peak load
burst size
request distribution
context distribution
output distribution
```

Then determine infrastructure requirements.

---

29. Architecture Decision Records

An architecture review should produce explicit decisions.

For example:

ADR-001 — Model provider **Decision:** Use hosted Model A for general requests and Model B for escalation.

Reason:

- lower expected cost
- acceptable quality
- B reserved for difficult cases

Rejected alternative: Always use Model B.

Tradeoff: Additional routing complexity.

This is an **Architecture Decision Record (ADR)**.

Other examples:

```
ADR-002: Vector database
ADR-003: Context management strategy
ADR-004: Tool authorization model
ADR-005: Model routing
ADR-006: Evaluation framework
ADR-007: Data retention
```

Architecture decisions should be recorded because future engineers otherwise see only the final implementation—not the reasoning behind it.

---

30. Architecture Review as a Decision Process

A useful review structure is:

```
1. Requirements
       ↓
2. Constraints
       ↓
3. Architecture
       ↓
4. Alternatives
       ↓
5. Tradeoffs
       ↓
6. Risks
       ↓
7. Decision
```

For each major decision ask:

> What alternatives did we consider?
>
> Why did we reject them?
>
> What assumptions does the decision depend on?
>
> When should we revisit it?

This is much stronger than:

> "This is how we implemented it."

---

31. The Architecture Review Document

The final deliverable should look something like:

```
Personal Research Assistant
Architecture Review
-----------------------------------

1. Executive Summary

2. Requirements

3. Architecture
   3.1 Logical architecture
   3.2 Deployment architecture
   3.3 Data flow
   3.4 Trust boundaries

4. API Specification

5. Data Model

6. Threat Model

7. Cost Model

8. Reliability Model

9. Evaluation Strategy

10. Performance Model

11. Architecture Decisions
```

`12. Risks and Mitigations`

`13. Open Questions`

`14. Capacity Assumptions`

`15. Test and Release Strategy`

This document should be something another engineer could use to understand and challenge the design without reading the implementation first.

---

32. The Architecture Review Meeting

Now simulate a real architecture review.

Have another engineer—or an AI agent—act as a skeptical reviewer.

The reviewer should ask:

Architecture

Why is this component separate?

Why is this state persistent?

Where are the boundaries?

Security

What happens if retrieved content contains malicious instructions?

Can the agent access another user's documents?

What prevents unauthorized tool calls?

Reliability

What happens if the model provider is unavailable?

What happens if retrieval times out?

What happens if the agent enters an infinite loop?

Performance

What is the P95 latency?

What is the bottleneck?

What happens at 10x traffic?

Economics

What is cost per successful task?

What happens if context length doubles?

Evaluation

How do you know the system is correct?

What happens when the model changes?

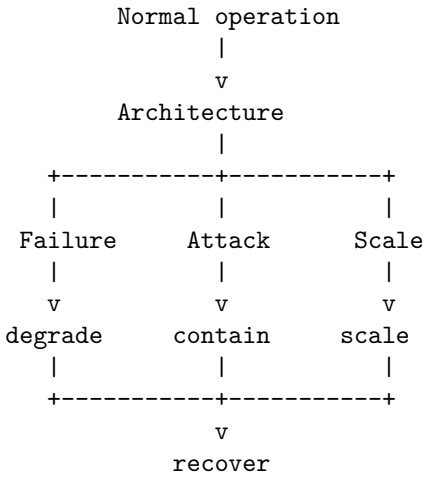
How do you detect behavioral regression?

A strong architecture should survive these questions.

---

33. Architecture Review Is About Failure

A useful mental model is:



The architecture is not primarily defined by what happens when everything works.

It is defined by what happens when:

- the model is unavailable,
- retrieval returns garbage,
- the database is slow,
- the tool fails,
- the user is malicious,
- the context is too large,
- the agent loops,
- traffic spikes,

- costs explode,
- the model changes,
- evaluation detects a regression.

This is why architecture review belongs after the reliability, security, performance, and testing chapters.

The individual disciplines now come together.

---

34. From Developer to Systems Engineer

A developer might think:

```
"I need to implement RAG."
```

An AI systems engineer thinks:

```
"What role does retrieval play in the system?"
```

A developer thinks:

```
"I need to call an LLM."
```

The systems engineer thinks:

```
"What is the model's reliability,
cost, latency, security boundary,
failure behavior, and evaluation contract?"
```

A developer thinks:

```
"The agent can call this API."
```

The systems engineer asks:

```
"Under what authorization model?
With what arguments?
With what limits?
What happens if the API fails?
What happens if the model is manipulated?"
```

A developer asks:

> "Does the feature work?"

The systems engineer asks:

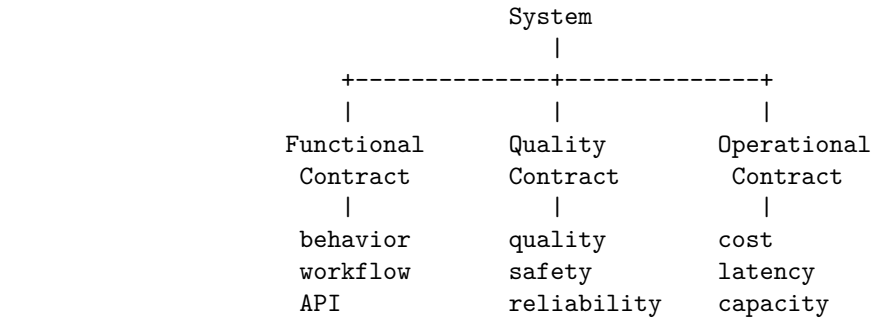
> **"Under what conditions does the system remain correct, safe, reliable, performant, and economically viable?"**

That is the transition.

---

35. The Architecture as a Contract

By the end of the review, you should be able to express the system as a collection of contracts:



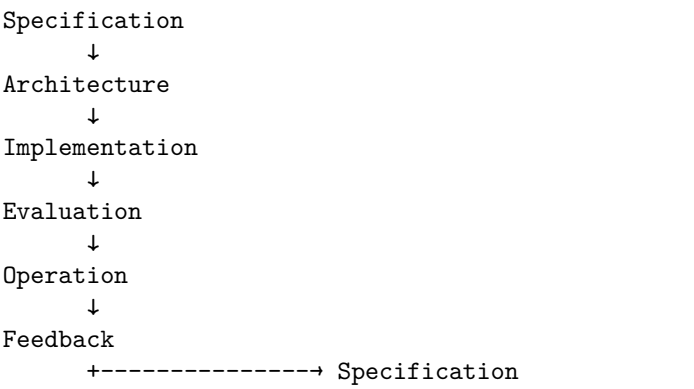
The architecture defines how those contracts are implemented.

The evaluation suite verifies them.

The observability system monitors them.

The deployment pipeline protects them.

This creates a coherent engineering discipline:



---

36. The Final Architecture Review Exercise

Take the Week 1 Personal Research Assistant and produce all eight artifacts.

1. Architecture diagram

Show:

- components
- data flows

- external dependencies
- trust boundaries
- model services
- tools
- storage
- observability

2. API specification

Define:

- endpoints
- request schemas
- response schemas
- authentication
- authorization
- errors
- limits
- versioning

3. Data model

Define:

- users
- documents
- chunks
- conversations
- messages
- tool executions
- evaluations
- permissions

4. Threat model

Identify:

- assets
- actors
- attack surfaces
- prompt injection
- indirect injection
- tool abuse
- data leakage
- supply-chain risks

5. Cost model

Estimate:

- requests/day
- tokens/request
- model calls/task
- model pricing
- storage
- compute
- observability
- cost per successful task

6. Reliability model

Define:

- failure domains
- SLOs
- retries
- timeouts
- fallbacks
- degraded modes
- agent limits
- disaster recovery

7. Evaluation strategy

Define:

- golden dataset
- regression suite
- quality metrics
- safety metrics
- retrieval metrics
- tool metrics
- performance metrics
- cost metrics
- release gates

8. Performance model

Define:

- latency budget
- throughput
- concurrency
- P50/P95/P99
- TTFT

- context limits
- GPU requirements
- bottlenecks
- scaling strategy

Then produce a final architecture decision:

> **Would you ship this system?**

If not:

> **What architectural changes are required before it is production-ready?**

---

37. Key Takeaways

1. **Architecture review is the transition from implementation to systems engineering.** The question changes from "Can I build it?" to "Can I defend the design?"
2. **Architecture is a set of explicit tradeoffs.** There is rarely a universally optimal design.
3. **Produce multiple architectural views.** Logical, deployment, dataflow, and security views expose different classes of problems.
4. **Every component needs a contract.** Define its responsibility, interface, dependencies, failure behavior, security boundary, scaling characteristics, and cost.
5. **APIs are not just schemas.** AI systems also require behavioral contracts describing what the system is expected to do.
6. **Separate persistent state from model context.** State is stored; context is deliberately constructed for each inference operation.
7. **Threat modeling must include the AI-specific attack surface.** Documents, retrieval results, tool outputs, and external content can all become indirect instruction channels.
8. **Cost belongs in the architecture.** Model calls, context length, retrieval, storage, compute, and observability all contribute to the economics of the system.
9. **Reliability must be modeled as a dependency graph.** The system should have explicit behavior for model failures, retrieval failures, tool failures, timeouts, and runaway agents.
10. **Graceful degradation is an architectural property.** A component failure should not automatically become a system failure.

11. **Evaluation is part of the architecture.** Regression suites, quality metrics, safety tests, and release gates should be integrated into the engineering lifecycle.

12. **Performance needs a quantitative model.** Define latency budgets, throughput, concurrency, token rates, capacity, and bottlenecks before optimizing.

13. **Architecture decisions should be recorded.** ADRs capture not only what was chosen but why alternatives were rejected.

14. **A serious architecture review is adversarial.** The reviewer should actively search for failures, attacks, bottlenecks, cost explosions, and hidden assumptions.

15. **The architecture should be evaluated under abnormal conditions.**

```
normal operation
     +
  failure
     +
  attack
     +
   scale
     =
production architecture
```

16. **The eight deliverables form one coherent model:**

```
Architecture
    |
    +-- API
    +-- Data
    +-- Security
    +-- Cost
    +-- Reliability
    +-- Evaluation
    +-- Performance
```

17. **The architecture is ultimately a set of guarantees and tradeoffs.** The implementation is merely one realization of those decisions.

18. **The defining transition is this:**

    A developer primarily reasons about **code**.

    An AI systems engineer reasons about **behavior across components, boundaries, failures, resources, and time**.

By the end of Chapter 14, the Week 1 application should no longer be thought of as a collection of Python modules, prompts, APIs, and database calls.

It should be possible to describe it as a **system with explicit contracts, boundaries, failure modes, economics, performance characteristics, and evaluation criteria**.

That is the point at which you stop merely building an AI application and begin **engineering an AI system**.